



COMSATS UNIVERSITY ISLAMABAD

Attock Campus

Department Of Computer Science

MUHAMMAD SUFYAN | FA24-BSE-049

Information Security LAB

Instructor: Mam Amreen Gull

Assignment :03



Submission date
May 2, 2026

TASK 1: Blockchain Implementation

Code:

```
import hashlib
import datetime

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = str(datetime.datetime.now())
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        value = str(self.index) + self.timestamp + self.data +
self.previous_hash
        return hashlib.sha256(value.encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    def add_block(self, data):
        prev_block = self.chain[-1]
        new_block = Block(len(self.chain), data, prev_block.hash)
        self.chain.append(new_block)

    def display(self):
        for block in self.chain:
            print("Index:", block.index)
            print("Data:", block.data)
            print("Hash:", block.hash)
            print("Prev Hash:", block.previous_hash)
            print("-" * 30)

# Test
bc = Blockchain()
bc.add_block("Block 1 Data")
bc.add_block("Block 2 Data")
bc.add_block("Block 3 Data")
bc.add_block("Block 4 Data")
```

```

bc.display()

# Modify a block
bc.chain[2].data = "Hacked Data"
bc.chain[2].hash = bc.chain[2].calculate_hash()

print("\nAfter Tampering:\n")
bc.display()

```

OUTPUT:

```

Active code page: 65001

D:\InformationSecurity>"C:\Program Files\Python313\python.exe" d:/InformationSecurity/InformationSecurity_task/IS_LAB_MTD.py/is_lab3.py/task1.py
Index: 0
Data: Genesis Block
Hash: 96129b00680bf652158b890468f1d9f8d08e31496044e4cd3610b7cdaa0b350f
Prev Hash: 0
-----
Index: 1
Data: Block 1 Data
Hash: 423ffa7f07593ce9e127ad042fc31725d5d17e930b32cbf36f778ac3c87a9cbf
Prev Hash: 96129b00680bf652158b890468f1d9f8d08e31496044e4cd3610b7cdaa0b350f
-----
Index: 2
Data: Block 2 Data
Hash: 3bd6fb33c4cfc3bc137f5a31bc0955479dded1920b4b12f77a3a440d9b86d3b8
Prev Hash: 423ffa7f07593ce9e127ad042fc31725d5d17e930b32cbf36f778ac3c87a9cbf
-----
Index: 3
Data: Block 3 Data
Hash: 5fabe925f1456cdd9b101a0454dd053c2a43cf9caf596aee59226262957cad50
Prev Hash: 3bd6fb33c4cfc3bc137f5a31bc0955479dded1920b4b12f77a3a440d9b86d3b8
-----
Index: 4
Data: Block 4 Data
Hash: de698832519ec3202f6f3591f783e876ae7c881cbeec8b28383002328ee05caa
Prev Hash: 5fabe925f1456cdd9b101a0454dd053c2a43cf9caf596aee59226262957cad50
-----

After Tampering:

Index: 0
Data: Genesis Block
Hash: 96129b00680bf652158b890468f1d9f8d08e31496044e4cd3610b7cdaa0b350f
Prev Hash: 0
-----
Index: 1
Data: Block 1 Data
Hash: 423ffa7f07593ce9e127ad042fc31725d5d17e930b32cbf36f778ac3c87a9cbf
Prev Hash: 96129b00680bf652158b890468f1d9f8d08e31496044e4cd3610b7cdaa0b350f
-----
Index: 2
Data: Hacked Data
Hash: 2472818038a7c7ee3b916a4a43715c82be59a14d651b14423fd3e7d2f32e8d2c
Prev Hash: 423ffa7f07593ce9e127ad042fc31725d5d17e930b32cbf36f778ac3c87a9cbf
-----
Index: 3
Data: Block 3 Data
Hash: 5fabe925f1456cdd9b101a0454dd053c2a43cf9caf596aee59226262957cad50
Prev Hash: 3bd6fb33c4cfc3bc137f5a31bc0955479dded1920b4b12f77a3a440d9b86d3b8
-----
Index: 4
Data: Block 4 Data
Hash: de698832519ec3202f6f3591f783e876ae7c881cbeec8b28383002328ee05caa
Prev Hash: 5fabe925f1456cdd9b101a0454dd053c2a43cf9caf596aee59226262957cad50
-----

```

TASK 2: ECC (Simplified)

Code:

```
p = 17
a = 2
b = 2

# Point Addition
def point_add(P, Q):
    if P == Q:
        return point_double(P)

    x1, y1 = P
    x2, y2 = Q

    m = ((y2 - y1) * pow(x2 - x1, -1, p)) % p
    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1 - x3) - y1) % p

    return (x3, y3)

# Point Doubling
def point_double(P):
    x, y = P

    m = ((3*x*x + a) * pow(2*y, -1, p)) % p
    x3 = (m*m - 2*x) % p
    y3 = (m*(x - x3) - y) % p

    return (x3, y3)

# Scalar multiplication
def scalar_mult(k, P):
    result = P
    for _ in range(k-1):
        result = point_add(result, P)
    return result

# Base point
G = (5, 1)

# Private key
d = 3
```

```

# Public key
Q = scalar_mult(d, G)

print("Private Key:", d)
print("Public Key:", Q)

# Simple Encryption (concept)
message = 2
k = 2

C1 = scalar_mult(k, G)
C2 = (message + k) % p

print("Cipher:", (C1, C2))

# Decryption
decrypted = (C2 - k) % p
print("Decrypted:", decrypted)

```

OUTPUT:

```

D:\informationSecuti>"C:\Program Files\Python313\python.exe" d:/informationSecuti/informationSecurity_ta
Private Key: 3py/is_lab3.py/task2.py
Public Key: (10, 6)
Cipher: ((6, 3), 4)
Decrypted: 2

D:\informationSecuti>

```